

ACTION AI · DESIGN TEAM

Reliable Design

The ActionAI Design Methodology

Version 1.0 · March 2026 · Owner: Tal (CPO), Ofer (Head of Design)

Why This Process Exists

Our product is built on one promise: Reliable Action AI. The system knows when it doesn't know. It never hallucinates its way through uncertainty — it explains, it surfaces, it earns trust.

Our design process holds the same standard.

The design team at ActionAI operates in a domain where mistakes carry real consequences. Our users — AI Engineers, Business Analysts, Admins working with regulated data in legal, healthcare, finance, and government — need a tool they can trust unconditionally. That trust is built one design decision at a time.

This document describes how we make those decisions. Not as a bureaucratic checklist, but as a shared operating system: a process the entire team — designers, PMs, engineers, and leadership — can understand, rely on, and hold each other accountable to.

Three things this process protects us from

- Jumping straight to Figma before we understand the problem. Visual polish is not the same as solved behavior. We prototype behavior first.
- Inconsistency across the team. Four designers working four different ways produce four different products. We ship one.
- Weak (and “too” late) handoffs to engineering. A design without a behavioral spec is an invitation for misinterpretation. Every shipped design comes with a PRD.

DESIGN IS A SCIENCE, NOT AN ART

We hypothesize, validate, and iterate. "I think users will prefer this" is a hypothesis — test it. The Fish Model and Double Diamond give us the framework to decide how much process each challenge actually deserves. We don't over-engineer small decisions, and we don't under-invest in strategic ones.

This process is also the first of its kind in our industry: a design workflow that uses AI — specifically Claude Code in VS Code — not as a generator, but as a planning and prototyping partner. Before a pixel lands in Figma, behavior is defined, coded, and validated. This is what "behavior first" looks like in practice.

The Process at a Glance

The Reliable Design method runs in six phases. The first two are grounded in the Problem Diamond — we discover and define the right thing to work on. Phases three through six are the Solution Diamond — we design, validate, and deliver the right solution.

Phase	Name	Owner	Deliverable
01	Input & Discovery	Whole team	Opportunity backlog
02	Prioritization + Classification	Design + PM	Scoped item + Track decision + User stories
03	Definition & AI Planning	Designer + PM	Planning brief (CC)
04	Prototype with Claude Code	Designer	Behavioral prototype
05	Testing & Alignment (& Figma)	Designer + Eng	Signed-off behavior
06	Deliver	Designer	FE to Dev –or– Figma + PRD

Two tracks — the fork at Phase 2

After Phase 2 classification, every piece of work takes one of two tracks. The track determines your tools, your timeline, and your deliverable. Commit to it explicitly.

Phase		TRACK A NEMO & TUNA · AI-First	TRACK B SALMON & WILLY · Full Process
01	Input & Discovery	— Same for both tracks	— Same for both tracks
02	Prioritization	Classify as Nemo / Tuna → take Track A	Classify as Salmon / Willy → take Track B
03	Definition	Planning session. Market research. If needed - run user interviews. Summarize into clear user stories.	User interviews + discovery methods + AI planning brief.
04	Prototype	Build directly with Claude Code.	Behavioral prototype with Claude Code — full state coverage.
05	Testing	Quick AI review + Eng sanity check.	Full round: AI, Engineering, PM, Users (where warranted).
06	Deliver	FE file → Dev. Lightweight behavioral notes. ** For Tuna we'll probably use Figma to organize "user flows"	Figma hi-fi (not always for Salmon) + complete Behavioral PRD. Willy: full flow organization required.
Typical duration		Nemo: 1 – 4 days Tuna: 1–2 weeks	Salmon: 2–3 weeks Willy: 3–4 weeks

The Double Diamond backbone

The Double Diamond is the structural backbone. Two diamonds, two problems to solve.

DIAMOND 1 · PROBLEM

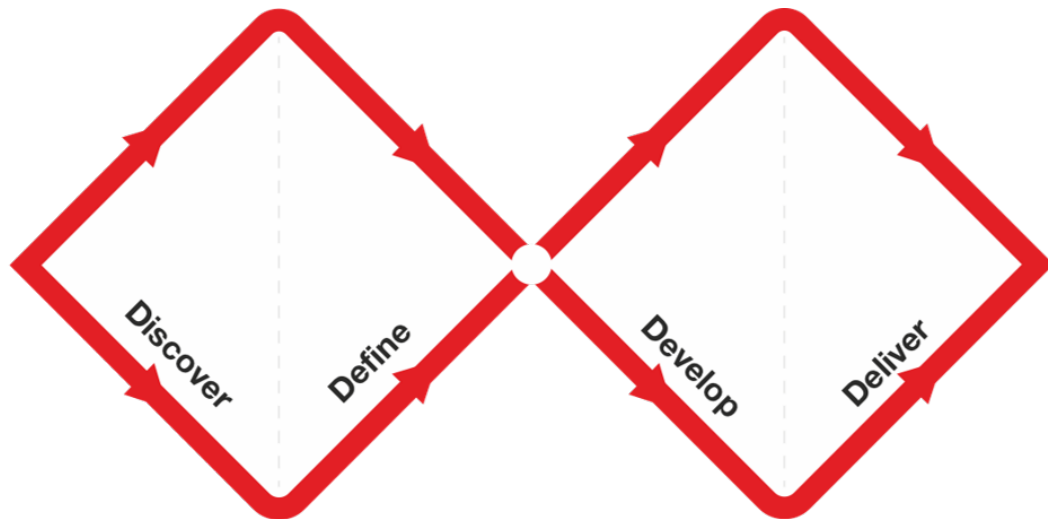
Are we solving the right problem?

Phases 01 – 02. Discover inputs, prioritize, define scope. We expand options (Discover) then narrow to a clear problem statement (Define).

DIAMOND 2 · SOLUTION

Are we solving it the right way?

Phases 03 – 06. Plan with AI, prototype behavior, test with engineering, finalize in Figma. We explore solutions (Design) then narrow to a validated, specified deliverable (Deliver).



The shape is intentional. Discover and Design phases diverge — we expand options, generate possibilities, resist premature closure. Define and Deliver phases converge — we make decisions, commit to specifics, ship. Teams that skip the Problem Diamond build elegant solutions to the wrong problem. Teams that never converge produce research that never influences anything.

The Six Phases

01 Input & Discovery

We maintain a continuous stream of signals about what to work on. No feature should emerge from someone's hunch — it should emerge from evidence.

INPUTS *Ongoing — all team members contribute*

OUTPUT *Identified opportunity backlog*

Signal sources

Design work starts with listening. The design team actively monitors four categories of inputs:

- **Users:** Direct user feedback, support tickets, session recordings, NPS responses, conversations with design partners (RAK Ceramics, RAK Courts, RAK Police, Morgan-Lewis).
- **UX Reviews:** Internal heuristic evaluations. Regular structured reviews where the design team audits flows against our seven design principles. These surface problems no user has articulated yet.
- **Tech Opportunities:** Engineering-identified improvements. When the platform gains a capability (new agent type, improved evaluation engine, faster rendering), design explores how to expose it well.
- **Competitive Signals:** Market movement, competitor launches, analyst reports. Not to copy, but to understand what patterns users may arrive expecting.

Every signal should be captured in a shared backlog. It's not a to-do list — it's a radar. The goal of Phase 1 is a rich, well-tagged pool of opportunities from which Phase 2 selects.

WHO CONTRIBUTES

Phase 1 is not just the design team's job. Engineering identifies tech debt with UX implications. PM surfaces customer feedback. Sales logs feature requests. Anyone who learns something about user behavior or product gaps should feed it into the signal backlog.

02 Prioritization & Track Selection

Not every signal deserves the same investment. Phase 2 decides what to work on AND which track to take. This is the fork in the road: Nemo and Tuna take the AI-First path; Salmon and Willy take the full process.

INPUTS *Prioritized opportunity backlog*

OUTPUT *Scoped work item + explicit Track A or Track B decision*

How we prioritize

We evaluate each item across four dimensions:

- **User impact:** How many users does this affect? How frequently? How severely?

- Business alignment: Does this directly serve a KPI — accuracy, time-to-creation, or orchestration efficiency?
- Effort signal: Is this a small, known fix or a large, uncertain exploration? This becomes the fish classification.
- Strategic timing: Is there a design partner demo, fundraiser, or sprint milestone this should serve?

Fish classification: the track decision

Two questions determine which fish you're dealing with — and therefore which track you take:

- How familiar are we? Do we know the domain, the users, the technology, and the relevant patterns?
- How wide is the scope? How many components, users, flows, and team dependencies are involved?

Fish	Familiarity	Scope	Track
 Nemo	High	Small	TRACK A — AI-First. No Figma. FE to Dev.
 Tuna	High	Large	TRACK A — AI-First. No Figma. FE to Dev.
 Salmon	Low	Small	TRACK B — Full process. Research first.
 Willy	Low	Large	TRACK B — Full process. Figma critical for flow coherency.

THE DISCIPLINE OF PHASE 2

The hardest thing about prioritization is not choosing what to do — it's choosing which track to commit to. A designer who mis-classifies a Willy as a Tuna will under-invest in research and Figma organization. A designer who treats every Nemo like a Willy will produce process theater. Classify honestly. Commit to the track.

03 Definition & AI Planning

We use user interviews and discovery methods to define the exact scope of this feature. Then — before any prototype is drawn — we use Claude Code in VS Code to plan the solution with AI as a collaborative partner.

INPUTS *Scoped work item from Phase 2*

OUTPUT *Planning brief (AI-generated, human-validated)*

User interviews and discovery

Before touching any tool, we talk to users. The method depends on scope and familiarity, but the goal is always the same: understand the user's mental model, current behavior, and the actual gap between where they are and where they need to be.

- For large, unfamiliar problems: structured user interviews, journey mapping, and Jobs-to-be-Done analysis.
- For mid-scope problems: task analysis, flow walkthroughs, heuristic review.
- For small, well-understood problems: a quick stakeholder check and competitive scan.

At the end of discovery, we write a clear problem statement: who the user is, what they're trying to accomplish, what's currently broken, and what a successful design outcome looks like.

Track A (Nemo & Tuna): Planning session, market research, and if needed, user interviews. Summarize into clear user stories. The AI planning brief is optional — use it when the problem has structural complexity.

Track B (Salmon & Willy): Full user interviews + discovery methods + AI planning brief. The planning brief is required and serves as the contract between designer and PM before prototyping begins.

AI Planning in VS Code with Claude Code

This is where our process diverges from the industry standard. Once the problem is defined, the designer and PM open VS Code with Claude Code loaded with the full ActionAI design context (DESIGN_CONTEXT.md) and begin a structured planning session.

This is not code generation. This is collaborative thinking. We use Claude Code to:

- Map the user flow and surface edge cases we might miss.
- Stress-test the problem definition — what are the failure modes? What are we not considering?
- Explore structural options before committing to any one approach.
- Generate a written planning brief that both PM and designer align on.

WHY PLAN IN CODE RATHER THAN FIGMA

Figma is a commitment. The moment something looks polished, teams treat it as a decision. Planning in text — in VS Code, with AI — keeps the conversation structural and behavioral. We're deciding what happens, not what it looks like. That's the right order.

Planning session structure

```
// Prompt structure for Phase 3 planning sessions
//
// Load DESIGN_CONTEXT.md first, then:

I am designing [feature name] for [user role: AI Engineer / BA / Admin].

The user is trying to: [goal in one sentence].
The current pain point is: [what is broken today].
Scope: [Nemo / Tuna / Salmon / Willy — based on Phase 2 sizing].

Help me:
1. Map the complete user flow including all edge cases and ExEx states.
2. Identify the 3 biggest design risks in this approach.
3. Define what "done" looks like for this feature.
4. Propose the information architecture for the screen.
```

04 Prototype with Claude Code

We prototype behavior — not visuals — using Claude Code in VS Code. The prototype runs in the browser. It is testable, shareable, and honest. It is not a Figma file.

INPUTS *Approved planning brief from Phase 3*

OUTPUT *Working behavioral prototype*

Why we prototype in code first

Most design teams prototype in Figma, then hand off to engineers, then discover that the behavior doesn't work as designed. We reverse this.

A code prototype forces us to specify behavior completely. You cannot fake an interaction state in code the way you can in a design tool. If a state is missing, the prototype breaks. That's the point — surface the gaps before engineering invests a single sprint.

- Code prototypes are testable. Engineers can run them. Users can interact with them.
- Code prototypes are honest. They show exactly what the behavior will be, not a polished impression of it.
- Code prototypes are fast to generate with AI. A working React component with full interaction states takes hours, not days.

What the prototype must include

A Phase 4 prototype is not a full production implementation. It must include:

- All primary interaction flows for the target user role.
- All empty states — the "nothing here yet" condition.

- All failure and ExEx states — what the user sees when something breaks.
- Role-appropriate permissions — what the target role can and cannot do.
- Cycle phase awareness — the user knows which phase of Build / Run / Evaluate / Improve they're in.

DEFINITION OF A DONE PROTOTYPE

A Phase 4 prototype is done when a designer who didn't build it can hand it to an engineer and say: "Build this. Every state is here." If any state requires imagination to fill in, the prototype is not done.

Prototyping prompt structure

```
// Load DESIGN_CONTEXT.md first, then:

Build a [screen/component name] for [user role].

The user is trying to: [goal from planning brief].
Reference pattern: [pattern from DESIGN_CONTEXT.md Section 5, if applicable].

Required states:
- Loading state
- Empty state (first use)
- Populated state (with realistic ActionAI data)
- Error state
- ExEx state (if applicable)

Constraints:
- Follow all design principles in DESIGN_CONTEXT.md
- Use only ActionAI design tokens
- Role: [AI Engineer / Business Analyst / Admin / Viewer]
```

05 Testing & Alignment

The prototype is tested — with AI, with engineers, and with users where scope warrants it. The goal is behavioral sign-off: everyone who needs to agree, agrees.

INPUTS Working behavioral prototype from Phase 4

OUTPUT Signed-off behavior, ready for Figma

Track A (Nemo & Tuna): Quick AI review + Engineering sanity check. PM review is optional unless the feature touches KPIs directly.

Track B (Salmon & Willy): Full review round — AI, Engineering, PM, and Users (where scope warrants). All four reviewer types in the table above apply.

Who tests, and what they're testing

Phase 5 testing is layered. Different reviewers answer different questions:

Reviewer	What they validate	Output
AI (Claude Code)	Completeness — are all states covered? Does the logic hold?	Structural feedback, missing edge cases
Engineering	Technical feasibility, performance implications, data model alignment	Build estimate + blockers
PM (Tal)	Behavioral alignment with KPIs and product strategy	Go / No-go to proceed
Users (when scope warrants)	Does the interaction match their mental model? Can they accomplish the task?	Usability findings

WHAT SIGN-OFF MEANS

Phase 5 ends with explicit alignment, not assumed agreement. Every stakeholder who reviewed the prototype either approves it or raises a specific, actionable concern. Silence is not approval. "Looks fine" is not sign-off. The designer logs the outcome and the decision before moving to Phase 6.

06 Deliver

Phase 6 has two endings depending on your track. Nemo and Tuna ship the FE file directly to dev (Tuna may add Figma for flow organization). Salmon and Willy go through the full Figma process. The bigger the fish, the more organized Figma needs to be — for Willy, Figma is the single source of truth for every flow, state, and dev handoff.

INPUTS Signed-off behavioral prototype from Phase 5

OUTPUT Track A: FE file → Dev · Track B: Figma + Behavioral PRD

TRACK A · NEMO & TUNA DELIVERY

The CC prototype is the deliverable. Figma is optional (Tuna may use it for flow organization).

For Nemo and Tuna-sized work, the behavioral prototype you built in Phase 4 — validated in Phase 5 — goes directly to the engineering team as the implementation reference. The FE code is the spec.

What the Nemo handoff includes (Tuna adds Figma for flow organization):

- The FE prototype file (React component or working HTML).
- Behavioral notes: 5–10 bullet points covering edge cases, ExEx states, and anything not obvious from the code.
- A Jira ticket with the scope, acceptance criteria, and link to the prototype.

Definition of done (Track A): An engineer can open the FE file, read the behavioral notes, and implement — without any design questions.

TRACK B · FULL PROCESS DELIVERY (Salmon / Willy)

Figma is the source of truth. The PRD is the contract.

The behavioral prototype becomes the reference for high-fidelity Figma work. The designer's job in Phase 6 is to bring visual precision to decisions that have already been validated — not to make new decisions.

Figma for Salmon

For Salmon-sized work, Figma provides the visual polish and the organized dev handoff. It must include:

- Every interaction state from the validated prototype — loading, empty, populated, error, ExEx.
- GenSys design system tokens applied throughout — no hardcoded colors or spacing.
- A final self-check against the seven ActionAI design principles.

Figma for Willy — flow organization is non-negotiable

Willy-scale work spans multiple flows, screens, and user roles. At this scale, Figma is not just a visual tool — it is the organizational document that makes engineering coherent. Without it, developers cannot understand how flows connect, how state transitions work across screens, or how the PRD maps to the interface.

WHY FIGMA IS CRITICAL AT WILLY SCALE

At Willy scale, the risk is not that a single screen is wrong — it's that the flows don't connect. A developer building Screen 4 without understanding Screen 2's state will create inconsistencies that break the user's mental model. Figma for Willy must show: all flows organized by user journey, explicit state transitions between screens, and clear PRD cross-references on each component. If a developer has to imagine how two flows connect, the Figma file is incomplete.

Willy Figma requirements:

- All user journeys mapped as linked flows — not individual screens.
- Every screen annotated with which PRD section it corresponds to.
- Cross-flow transitions explicitly shown — not implied.
- A cover page in the Figma file that orients any developer to the overall structure before they dive into details.

The Behavioral PRD — required for all Track B deliveries (Salmon & Willy)

Before any Track B Figma file is handed to engineering, the designer writes a Behavioral PRD. Track A deliveries (Nemo & Tuna) use behavioral notes instead.

Section	What it contains
User & Role Context	Who this feature serves, what role they hold, what they can and cannot do
Success Scenario	The primary user goal — what they're trying to accomplish and what success looks like
Complete State Inventory	Every state the UI can be in: loading, empty, populated, error, ExEx, disabled, forbidden
Interaction Specifications	What happens on every click, tap, and keyboard action — no ambiguity
Edge Cases & Exceptions	What the user sees when something goes wrong — ExEx rules, error messages, fallback states
Copy & Microcopy	All UI text: labels, placeholders, error messages, empty state copy, confirmation text
Acceptance Criteria	The conditions under which engineering considers this feature done

DEFINITION OF DONE — TRACK B

A feature is ready to hand to engineering when: (1) The Figma file contains every state from the validated prototype. For Willy: all flows are linked and cross-referenced to the PRD. (2) The Behavioral PRD is complete and reviewed by PM. (3) An engineer can implement the feature without needing to ask the designer a single question about behavior.

What We Don't Do

These are the failure modes this process is designed to prevent. Every designer on the team should be able to call these out — in their own work and in code reviews.

✗ DO NOT Open Figma before writing a problem statement.	✓ INSTEAD Write the problem statement first. Define who the user is, what they're trying to do, and what's broken. Then plan. Then prototype. Then Figma.
✗ DO NOT Treat discovery as optional for complex work.	✓ INSTEAD Discovery depth scales with scope and unfamiliarity. Small, well-understood problems need less. Large or novel challenges demand structured research. Skipping it on a Willy-scale problem is how we ship the wrong thing beautifully.
✗ DO NOT Design only the success state.	✓ INSTEAD Every screen must have a designed empty state, error state, and ExEx state. Failure states are first-class citizens in ActionAI — they are core to the product promise.
✗ DO NOT Hand Track B Figma files to engineering without a Behavioral PRD.	✓ INSTEAD A design file without behavioral specs is an invitation for misinterpretation. Every Track B handoff includes a PRD. Track A (Nemo & Tuna) uses behavioral notes instead.
✗ DO NOT Skip Phase 5 alignment because "everyone is aligned."	✓ INSTEAD Assumed alignment is not alignment. Get explicit sign-off from engineering and PM before investing in Figma polish. Discovering a behavioral flaw after hi-fi design is expensive.
✗ DO NOT Introduce new visual patterns without documenting them.	✓ INSTEAD Every new pattern added to the product must be documented in DESIGN_CONTEXT.md before shipping. Consistency is a feature. Undocumented one-offs compound into a system no one controls.

Appendix: AI Tooling Guide

This appendix is the hands-on reference for the AI-augmented phases of our process (Phases 3 and 4). It explains exactly how to set up and run effective planning and prototyping sessions with Claude Code in VS Code.

Setup

Before every session

- Open VS Code in the ActionAI project root.
- Load DESIGN_CONTEXT.md into the Claude Code context window first — this is the single most important step. Without it, Claude Code has no knowledge of ActionAI's design system, principles, or patterns.
- Have the relevant Figma file open for reference if working on an existing screen.
- Have the Phase 2 scoped work item and Phase 3 problem statement visible.

CRITICAL: CONTEXT LOADING

Claude Code's output quality is directly proportional to the context it receives. Always begin with: "Load DESIGN_CONTEXT.md. Acknowledge when ready." Do not start generating anything until Claude Code confirms it has processed the full file.

Phase 3: Planning Session Playbook

A well-run Phase 3 planning session should take 60–90 minutes and produce a written brief that PM and designer both sign off on. Run it in this order:

- Step 1 — Context load. Confirm DESIGN_CONTEXT.md is in context.
- Step 2 — Problem statement. Share the problem statement from discovery. Ask Claude Code to identify any gaps or unstated assumptions.
- Step 3 — Flow mapping. Ask Claude Code to map the complete user flow, including all states and edge cases. Review and challenge the output.
- Step 4 — Risk surfacing. Ask: "What are the 3 most likely design mistakes we could make on this feature?" Use the response to stress-test your approach.
- Step 5 — Structure options. Ask for 2–3 different structural approaches to the screen. Evaluate each against ActionAI design principles.
- Step 6 — Write the brief. Synthesize the session into a written planning brief. This is the input to Phase 4.

Phase 4: Prototyping Session Playbook

The prototyping session translates the planning brief into a working React prototype. Approach it in components, not all at once:

- Generate one component or state at a time.
- Review each output against the seven design principles before proceeding.
- When a state is missing or a pattern is wrong, fix it before moving forward — do not defer.
- Run the prototype in the browser and interact with it yourself before sharing it for review.

Designer review checklist after each generation

☐	Uses correct ActionAI design tokens — no hardcoded colors or spacing
☐	ExEx / failure state is designed
☐	Follows the density principle — no excessive whitespace
☐	Role-based permissions are applied correctly
☐	User knows which cycle phase they're in (Build / Run / Evaluate / Improve)
☐	Empty state is designed
☐	No new undocumented patterns introduced
☐	Would I hand this to an engineer with confidence?

ActionAI Design Team · Reliable Design v1.0 · March 2026

Owner: Tal (CPO) · Update Cadence: Every 2 sprints